



DOCUMENTACIÓN TÉCNICA

PokemonGame

Curso:

Diseño de Patrones

Grupo: 01

Docente:

Condori Bustincio Romulo Walter

INTEGRANTES:

1. Chura Benavente Gerardo Gonzalo
2. Guardia Quispe Luis Bryan
3. Lopez Calderon Andrea del Rosario
4. Paucar Choquehuanca Marco Antonio

Arequipa-Peru

2026

Contenido

1. Introducción	3
2. Arquitectura del sistema	3
2.1 Visión general.....	3
2.2 Estructura de paquetes.....	3
2.3 Flujo general de ejecución	4
3. Patrones de diseño (GoF) identificados	5
3.1 Patrones creacionales	5
3.1.1 Builder	5
3.1.2 Singleton	6
3.1.3 Prototype.....	6
3.2 Patrones estructurales	7
3.2.1 Facade	7
3.2.2 Composite.....	7
3.3 Patrones de comportamiento.....	8
3.3.1 State	8
3.3.2 Observer	9
3.3.3 Command	10
3.3.4 Memento	11
3.3.5 Strategy.....	11
4. Principios SOLID — evidencias en el código	12
4.1 SRP — Principio de Responsabilidad Única.....	12
4.2 OCP — Principio de Abierto/Cerrado	13
4.3 LSP — Principio de Sustitución de Liskov.....	13
4.4 ISP — Principio de Segregación de Interfaces.....	14
4.5 DIP — Principio de Inversión de Dependencias.....	14
5. Principios GRASP — evidencias en el código	16
5.1 Experto en información (Information Expert).....	16
5.2 Creador (Creator)	16
5.3 Bajo acoplamiento (Low Coupling) y Alta cohesión (High Cohesion)	17
5.4 Controlador (Controller)	17
5.5 Polimorfismo (Polymorphism).....	17
5.6 Fabricación pura (Pure Fabrication)	17
5.7 Variación protegida (Protected Variation).....	18
6. Tabla de Antipatrones evitados	18
7. Conclusiones	19

1. Introducción

El presente documento describe la arquitectura técnica del proyecto PokemonGame, una aplicación de escritorio desarrollada en Java (Swing) que simula combates por turnos entre entrenadores Pokémon, con persistencia de datos en PostgreSQL. El objetivo de este informe es analizar el código fuente del proyecto para identificar de forma objetiva los patrones de diseño de la Banda de los Cuatro (GoF) efectivamente implementados, así como recopilar evidencias concretas de la aplicación de los principios SOLID y GRASP, tomando como base directa las clases del código y no una descripción teórica genérica.

2. Arquitectura del sistema

2.1 Visión general

PokemonGame es una aplicación monolítica de escritorio construida con Maven (pom.xml) que sigue una separación por capas dentro de un único módulo. La capa de presentación (Swing) nunca accede directamente al modelo de datos ni a la base de datos: toda comunicación pasa por una fachada (PokemonGameFacade) o por objetos de dominio expuestos explícitamente (Entrenador, Combate). La capa de persistencia se aísla en el paquete SQL_Conexion, y la lógica de negocio de combate se concentra en Combate.Atk.

2.2 Estructura de paquetes

Paquete	Responsabilidad	Clases principales
com.mycompany.pokemongame	Punto de entrada de la aplicación (main).	Main
com.mycompany.gui	Capa de presentación (Swing). Ventanas, paneles y adaptadores de patrones para la UI.	VentanaPrincipal, VentanaBatalla, VentanaSeleccionEquipo, PanelBatalla, PanelPokedex, SelectorAtaqueGUI, SelectorAtaqueIA, CombateObservadorGUI, RecursosImagenes, EstiloJuego
com.mycompany.Facade	Fachada única entre la GUI y el resto del sistema	PokemonGameFacade
com.mycompany.Model.pokemon	Entidad de dominio Pokémon y su enum de tipo.	Pokemon, TipoPokemon
com.mycompany.Model.entrenador	Entidad Entrenador, gestión de equipo, mochila y estrategia de selección de ataque.	Entrenador, SelectorAtaque
com.mycompany.Model.pokedex	Registro global de Pokémon disponibles (catálogo).	Pokedex
com.mycompany.Combate.Atk	Motor de batalla: ataques, cálculo de daño, tabla de	Combate, Ataque, AtaqueFisico, AtaqueEspecial,

Paquete	Responsabilidad	Clases principales
	tipos, turnos, observadores y snapshot del combate.	AtaqueComand, Calculadora, TablaTipos, CombateObservador, BitacoraCombate, ConsolaObservador, CombateMemento
com.mycompany.Patrones.state	Implementación explícita del patrón State para estados alterados del Pokémon.	EstadoPokemon, EstadoNormal, EstadoDormido, EstadoParalizado, EstadoQuemado
com.mycompany.Patrones.composite	Implementación explícita del patrón Composite para la mochila del entrenador.	ItemMochila, Pocion, MochilaGrupo
com.mycompany.Patrones.prototype	Contrato genérico del patrón Prototype, reutilizado por varias entidades de dominio.	Prototype<T>
com.pokemon.builder	Builder para construir instancias de Pokemon a partir de datos sueltos o de la base de datos.	PokemonBuilder
com.mycompany.SQL_Conexion	Persistencia: conexión JDBC a PostgreSQL y consultas de Pokémon.	DatabaseConnection, PokemonDBQ

2.3 Flujo general de ejecución

El flujo de la aplicación, reconstruido a partir de Main.java, es el siguiente:

- Main.iniciarJuego() obtiene el catálogo de Pokémon a través del Singleton Pokedex, el cual internamente delega en PokemonDBQ para poblarse desde PostgreSQL usando PokemonBuilder.
- Se abre VentanaSeleccionEquipo para que el jugador arme su equipo (hasta 3 Pokémon) eligiendo entre los disponibles en la Pokédex.
- Main.iniciarCombate(...) construye dos objetos Entrenador (jugador y rival), clona los Pokémon base (patrón Prototype) y les agrega ataques armados con el Builder de Ataque.
- Se crea un objeto Combate, se le registra un observador de bitácora (BitacoraCombate) y se abre VentanaBatalla, que añade además un observador gráfico (CombateObservadorGUI).
- Durante la batalla, Combate coordina los turnos, delega en Entrenador.elegirAtaque(...) (patrón Strategy) y ejecuta cada ataque como un objeto Comand (AtaqueComand), notificando a todos los observadores registrados (patrón Observer).

Esta separación permite que la lógica de combate (Combate, Ataque, Calculadora) sea completamente independiente de si el ataque lo elige un humano por consola, un humano por clics en Swing o una IA, y de si el resultado del turno se muestra en consola, en la interfaz gráfica, o en ambas simultáneamente.

3. Patrones de diseño (GoF) identificados

A continuación, se documentan los patrones de diseño de la Banda de los Cuatro (Gang of Four) identificados en el código fuente, organizados según su categoría clásica: creacionales, estructurales y de comportamiento. Cada patrón incluye las clases donde se implementa, su ubicación en el proyecto y la evidencia textual (fragmento de código o justificación basada en la implementación real) que sustenta su identificación.

3.1 Patrones creacionales

3.1.1 Builder

El patrón Builder se utiliza en dos puntos distintos del proyecto para separar la construcción de objetos complejos (Ataque y Pokemon) de su representación final, permitiendo construirlos mediante una interfaz fluida encadenada.

Elemento	Ubicación	Evidencia
Ataque.Builder<T> (Builder genérico anidado)	Combate/Atk/Ataque.java	Clase abstracta interna con generics auto-referenciados (T extends Builder<T>) y método abstracto self(), que permite que AtaqueFisicoBuilder y AtaqueEspecialBuilder encadenen métodos propios sin perder el tipo concreto del builder.
AtaqueFisico.AtaqueFisicoBuilder	Combate/Atk/AtaqueFisico.java	Extiende Ataque.Builder<AtaqueFisicoBuilder>; construye AtaqueFisico con nombre, tipo y potencia.
AtaqueEspecial.AtaqueEspecialBuilder	Combate/Atk/AtaqueEspecial.java	Añade el método propio efectoEstado(EstadoPokemon, int) además de los heredados del builder base, antes de invocar build().
PokemonBuilder	Patrones/builder/PokemonBuilder.java (paquete com.pokemon.builder)	Métodos conId(), conNombre(), conTipo(), conHp(), etc., cada uno retorna this; build() ensambla el Pokemon final y le asigna el estado inicial (integración con el patrón State).

```
public static abstract class Builder<T extends Builder<T>> {  
    public T potencia(int potencia) { this.potencia = potencia; return self(); }  
    public abstract Ataque build();  
    protected abstract T self();  
}
```

Uso concreto en *PokemonDBQ*, donde cada fila del *ResultSet* se traduce a un *Pokemon* completamente construido mediante el builder, evitando un constructor con nueve parámetros posicionales:

```
return new PokemonBuilder()  
    .conId(rs.getInt("p_id"))  
    .conNombre(rs.getString("p_nombre"))  
    .conTipo(TipoPokemon.valueOf(rs.getString("p_tipo")).toUpperCase())
```

```
...  
.build();
```

3.1.2 Singleton

Se identificaron tres implementaciones del patrón Singleton, todas con constructor privado y método estático de acceso `getInstancia()`:

Clase	Ubicación	Evidencia
TablaTipos	Combate/Atk/TablaTipos.java	Constructor privado; <code>getInstancia()</code> sincronizado (<code>synchronized</code>) con inicialización perezosa (<code>lazy</code>). Guarda la tabla de efectividad entre tipos, un dato que debe existir una única vez en memoria.
Calculadora	Combate/Atk/Calculadora.java	Constructor privado; <code>getInstancia()</code> sin sincronización. Centraliza las fórmulas de daño físico y especial.
Pokedex	Model/pokedex/Pokedex.java	Constructor privado que internamente instancia <code>PokemonDBQ</code> y carga la lista completa desde la base de datos; <code>getInstancia()</code> garantiza una única carga del catálogo durante la ejecución.

```
public static synchronized TablaTipos getInstancia(){  
    if (instancia == null) { instancia = new TablaTipos(); }  
    return instancia;  
}
```

3.1.3 Prototype

El patrón Prototype se formaliza mediante una interfaz genérica reutilizable, `Prototype<T>`, implementada por tres jerarquías distintas del dominio. Esto permite clonar objetos complejos sin acoplar el código cliente a sus clases concretas.

Clase / interfaz	Ubicación	Evidencia
<code>Prototype<T></code>	Patrones/prototype/Prototype.java	Interfaz raíz del patrón: único método <code>T clonar()</code> .
Ataque (y subclases)	Combate/Atk/Ataque.java	<code>clonar()</code> abstracto; <code>AtaqueFisico</code> y <code>AtaqueEspecial</code> lo implementan reconstruyendo el ataque vía su propio Builder (integración Prototype + Builder).
EstadoPokemon (y estados concretos)	Patrones/state/EstadoPokemon.java	La interfaz extiende <code>Prototype<EstadoPokemon></code> ; <code>EstadoDormido.clonar()</code> preserva incluso el atributo mutable <code>turnosRestantes</code> , evitando que dos Pokémon

Clase / interfaz	Ubicación	Evidencia
		compartan la misma instancia de estado.
Pokemon	Model/pokemon/Pokemon.java	clonar() crea una nueva instancia con los mismos stats base, pero con hpActual = hp y estado = EstadoNormal, evitando que el clon herede el HP dañado o el estado alterado del original.
Entrenador	Model/entrenador/Entrenador.java	clonar() combina Prototype con Cloneable (super.clone()) y clona en profundidad el arreglo de Pokemon (clonando cada Pokemon individualmente), usado por Combate para guardar una foto exacta del estado inicial (ver Memento).

```
// Pokemon.clonar()
clon.hpActual = this.hp;
clon.estado = new EstadoNormal();
clon.ataques.addAll(this.ataques);
return clon;
```

3.2 Patrones estructurales

3.2.1 Facade

PokemonGameFacade centraliza el acceso de la capa GUI al resto del sistema. Su propio comentario Javadoc declara explícitamente la intención del patrón: "La GUI nunca debe hablar directamente con Pokedex, Entrenador, Combate, etc. Todo pasa por esta clase". Actualmente expone dos operaciones de consulta (obtenerTodosLosPokemon, buscarPokemon), delegando en el Singleton Pokedex.

Elemento	Ubicación	Evidencia
PokemonGameFacade	Facade/PokemonGameFacade.java	Constructor obtiene la instancia única de Pokedex; los métodos públicos son la única superficie que la GUI necesita conocer, ocultando la existencia del Singleton y de PokemonDBQ.

3.2.2 Composite

Rol Composite	Clase	Ubicación	Evidencia
Componente	ItemMochila	Patrones/composite/ItemMochila.java	Declara getNombre(), getCantidad() y

Rol Composite	Clase	Ubicación	Evidencia
			usar(Pokemon), sin distinguir si el implementador es una hoja o un grupo.
Hoja	Pocion	Patrones/composite/Pocion.java	Ítem individual (Poción, Súper Poción, Hiper Poción) con cantidad y HP curado propios; no contiene otros ItemMochila.
Compuesto	MochilaGrupo	Patrones/composite/MochilaGrupo.java	Contiene una List<ItemMochila>; getCantidad() suma recursivamente (stream + mapToInt) la cantidad de todos sus hijos; tienelItem(...) busca recursivamente en subgrupos.

La mochila del entrenador se modela como un árbol Composite, donde tanto un ítem individual (una Poción) como un grupo de ítems (una categoría de mochila, o la mochila completa) se tratan de forma uniforme a través de la interfaz ItemMochila.

```
// MochilaGrupo — suma recursiva de cantidades
public int getCantidad() {
    return items.stream().mapToInt(ItemMochila::getCantidad).sum();
}
```

El propio Javadoc de Entrenador y de ItemMochila referencia explícitamente los principios SOLID que motivan este diseño (LSP e ISP), lo cual se retoma en la sección 4.

3.3 Patrones de comportamiento

3.3.1 State

El comportamiento de un Pokémon durante su turno varía según su estado alterado (Normal, Dormido, Paralizado, Quemado). En lugar de condicionales dentro de Pokemon, cada estado concreto implementa su propia lógica de allniciarTurno(...) y alFinalTurno(...), y Pokemon delega en el objeto estado actual.

Clase	Comportamiento distintivo	Ubicación
EstadoPokemon (interfaz)	Contrato con getNombre(), allniciarTurno(), alFinalTurno() y modificarVelocidad() (con implementación por defecto).	Patrones/state/EstadoPokemon.java
EstadoNormal	No altera nada; siempre permite atacar.	Patrones/state/EstadoNormal.java
EstadoDormido	Bloquea el ataque entre 1 y 3 turnos (aleatorio) y se auto-transiciona a EstadoNormal al despertar	Patrones/state/EstadoDormido.java

Clase	Comportamiento distintivo	Ubicación
	(pokemon.setEstado(new EstadoNormal())).	
EstadoParalizado	25% de probabilidad de no poder moverse cada turno; reduce la velocidad a la mitad vía modificarVelocidad().	Patrones/state/EstadoParalizado.java
EstadoQuemado	Resta 1/16 del HP máximo al final de cada turno, sin impedir atacar.	Patrones/state/EstadoQuemado.java
Pokemon	iniciarTurno() y finalTurno() delegan sin condicionales: return estado.alIniciarTurno(this);	Model/pokemon/Pokemon.java

```
// Pokemon — delegación pura al patrón State
public boolean iniciarTurno() { return estado.alIniciarTurno(this); }
public void finalTurno() { estado.alFinalTurno(this); }
```

3.3.2 Observer

Combate actúa como sujeto (subject) observable: mantiene una lista de CombateObservador y notifica cada evento relevante de la batalla (cambio de turno, ataque realizado, Pokémon debilitado, cambio de Pokémon, fin del combate) a todos los observadores registrados, sin conocer sus implementaciones concretas.

Rol Observer	Clase	Ubicación	Evidencia
Sujeto	Combate	Combate/Atk/Combate.java	Mantiene List<CombateObservador> observadores; registrarObservador(o) permite añadir observadores en tiempo de ejecución; los métodos notificar*() iteran la lista invocando el callback correspondiente.
Observador (interfaz)	CombateObservador	Combate/Atk/CombateObservador.java	Declara onCambioTurno, onAtaqueRealizado, onPokemonDebilitado, onPokemonCambiado, onCombateTerminado.
Observador concreto	BitacoraCombate	Combate/Atk/BitacoraCombate.java	Traduce cada evento a mensajes de consola con formato de bitácora, incluyendo el estado alterado actual del Pokémon.
Observador concreto	ConsolaObservador	Combate/Atk/ConsolaObservador.java	Solo reacciona a onCombateTerminado, ofreciendo reiniciar el combate mediante

Rol Observer	Clase	Ubicación	Evidencia
			Combate.reiniciarDesdeInicio().
Observador concreto	CombateObservador GUI	GUI/CombateObservadorGUI.java	Traduce los mismos eventos en actualizaciones visuales sobre PanelBatalla, despachadas al Event Dispatch Thread de Swing con SwingUtilities.invokeLater().

```
@Override
public void notificarAtaque(Pokemon atacante, Pokemon defensor, String ataque, int damage) {
    for (CombateObservador o : observadores) {
        o.onAtaqueRealizado(atacante, defensor, ataque, damage);
    }
}
```

3.3.3 Command

Cada ataque ejecutado dentro de un turno se encapsula como un objeto Comand independiente (AtaqueComand), que agrupa al receptor (Ataque), a los participantes (atacante, defensor) y al invocador (Combate, a través de la interfaz CombateCallback), permitiendo que Combate desencadene la acción sin conocer su lógica interna.

Rol Command	Clase	Ubicación	Evidencia
Comando (interfaz)	Comand	Combate/Atk/Comand.java	Un único método: void ejecutar().
Comando concreto	AtaqueComand	Combate/Atk/AtaqueComand.java	Encapsula ataque, atacante, defensor, defensorEnt y callback; ejecutar() invoca al receptor (ataque.atacar(...)), calcula el daño aplicado y notifica al invocador mediante CombateCallback.
Receptor	Ataque	Combate/Atk/Ataque.java	Clase abstracta con el comportamiento real que el comando termina disparando (atacar()).
Invocador / Receptor de notificaciones	Combate	Combate/Atk/Combate.java	ejecutarAccion(...) crea new AtaqueComand(...) y llama a cmd.ejecutar(); Combate implementa CombateCallback para recibir las

Rol Command	Clase	Ubicación	Evidencia
			notificaciones del comando.

```
// Combate.ejecutarAccion — construcción e invocación del Command
AtaqueComand cmd = new AtaqueComand(ataqueElegido, atacante, defensor, defensorEnt, this);
cmd.ejecutar();
```

3.3.4 Memento

Combate implementa un mecanismo de snapshot/restore del estado inicial de la batalla, apoyándose en el patrón Prototype para clonar en profundidad a ambos entrenadores. Esto permite reiniciar un combate perdido exactamente en las condiciones con las que empezó.

Rol Memento	Clase	Ubicación	Evidencia
Memento	CombateMemento	Combate/Atk/CombateMemento.java	Clase final e inmutable que guarda turnoGeneral y la lista de entrenadoresClonados.
Originador (Originator)	Combate	Combate/Atk/Combate.java	guardarEstadoInicial() crea el memento clonando entrenador1.clonar() y entrenador2.clonar(); restaurarEstadoInicial() restablece turnoGeneral y los entrenadores desde el memento guardado.
Cuidador (Caretaker)	ConsolaObservador	Combate/Atk/ConsolaObservador.java	Al recibir onCombateTerminado(...), si el jugador pierde, invoca combate.reiniciarDesdeInicio(), que internamente llama a restaurarEstadoInicial() seguido de iniciarBatalla().

3.3.5 Strategy

La forma en que un entrenador decide qué ataque usar en su turno se extrae a una interfaz intercambiable, SelectorAtaque, desacoplando por completo a Entrenador y Combate de si la elección proviene de la consola, de clics en la GUI o de una inteligencia artificial simple.

Estrategia	Clase	Ubicación	Evidencia
Interfaz de estrategia	SelectorAtaque	Model/entrenador/SelectorAtaque.java	Ataque elegir(Pokemon activo, List<Ataque> ataques); Javadoc declara explícitamente el uso del patrón Strategy.
Estrategia concreta (consola)	SelectorAtaqueConsola	Model/entrenador/Entrenador.java (clase interna privada)	Estrategia por defecto de Entrenador; usa Scanner para leer la opción del jugador desde consola.

Estrategia	Clase	Ubicación	Evidencia
Estrategia concreta (GUI humano)	SelectorAtaqueGUI	GUI/SelectorAtaqueGUI.java	Bloquea el hilo de combate (no el EDT) hasta que el jugador hace clic en un botón de ataque o usa un ítem de la mochila.
Estrategia concreta (IA)	SelectorAtaqueIA	GUI/SelectorAtaqueIA.java	Introduce una pausa aleatoria y elige un ataque al azar de la lista disponible, simulando al rival.
Contexto	Entrenador	Model/entrenador/Entrenador.java	setSelectorAtaque(...) permite intercambiar la estrategia en tiempo de ejecución; elegirAtaque(...) delega siempre en selectorAtaque.elegir(...).

4. Principios SOLID — evidencias en el código

Esta sección recoge, principio por principio, evidencia concreta —clase, archivo y justificación— de cómo el proyecto aplica los cinco principios SOLID.

4.1 SRP — Principio de Responsabilidad Única

Clase	Ubicación	Responsabilidad única evidenciada
Calculadora	Combate/Atk/Calculadora.java	Únicamente calcula fórmulas de daño (físico y especial); no conoce reglas de turno, ni de estados, ni de UI.
TablaTipos	Combate/Atk/TablaTipos.java	Únicamente almacena y consulta la matriz de efectividad entre tipos (getMultiplicador); no calcula daño ni ejecuta ataques.
PokemonDBQ	SQL_Conexion/PokemonDBQ.java	Se ocupa únicamente de traducir entre los objetos del dominio (Pokemon) y las tablas de la base de datos; no contiene reglas de negocio de combate.
BitacoraCombate	Combate/Atk/BitacoraCombate.java	Su única razón de cambio es el formato del registro/log de eventos de combate en consola.
DatabaseConnection	SQL_Conexion/DatabaseConnection.java	Únicamente gestiona la obtención de la conexión JDBC; no ejecuta consultas ni mapea resultados (eso lo hace PokemonDBQ).

4.2 OCP — Principio de Abierto/Cerrado

Punto de extensión	Ubicación	Evidencia
Nuevos estados alterados	Patrones/state/*	Agrega un nuevo estado (ej. Congelado) = nueva clase, sin modificar Pokemon ni los estados existentes". Pokemon solo depende de la interfaz EstadoPokemon.
Nuevos tipos de ítems de mochila	Patrones/composite/*	El Javadoc de MochilaGrupo indica: "se pueden agregar nuevos tipos de ítems sin modificar esta clase"; basta con implementar ItemMochila.
Nuevos tipos de ataque	Combate/Atk/Ataque.java y subclases	El Builder genérico Ataque.Builder<T> permite añadir un nuevo tipo de ataque (nueva subclase de Ataque + su propio Builder) sin modificar Ataque ni los tipos existentes.
Nuevas estrategias de selección de ataque	Model/entrenador/SelectorAtaque + implementaciones	SelectorAtaqueGUI y SelectorAtaqueIA se añadieron sin tocar Entrenador ni Combate, únicamente implementando la interfaz existente.

4.3 LSP — Principio de Sustitución de Liskov

Jerarquía	Ubicación	Evidencia
EstadoPokemon	Patrones/state/*	Pokemon.iniciarTurno()/finalTurno() invocan la interfaz sin importar la implementación concreta; cualquier EstadoPokemon (Normal, Dormido, Paralizado, Quemado) es sustituible sin romper el contrato (el propio Javadoc lo declara: "cualquier EstadoPokemon es intercambiable en Pokemon").
ItemMochila	Patrones/composite/*	MochilaGrupo.usar(...) invoca item.usar(objetivo) sobre cada hijo sin distinguir si es una Poción (hoja) u otro MochilaGrupo (compuesto); ambos cumplen el mismo contrato.
SelectorAtaque	Model/entrenador/* y GUI/*	Entrenador.elegirAtaque(...) delega en selectorAtaque.elegir(...) sin ningún cambio de comportamiento según sea SelectorAtaqueConsola, SelectorAtaqueGUI o SelectorAtaqueIA.
Ataque	Combate/Atk/Ataque.java	AtaqueComand.ejecutar() llama a ataque.atacar(...) de forma polimórfica; AtaqueFisico y AtaqueEspecial son

Jerarquía	Ubicación	Evidencia
		sustituibles entre sí desde la perspectiva del comando.

4.4 ISP — Principio de Segregación de Interfaces

Interfaz	Ubicación	Evidencia
ItemMochila	Patrones/composite/ItemMochila.java	Expone solo tres métodos (getNombre, getCantidad, usar); el Javadoc de Entrenador lo confirma: "el Entrenador solo ve usar() y getNombre(), no la estructura" interna del árbol Composite.
SelectorAtaque	Model/entrenador/SelectorAtaque.java	Interfaz de un único método (elegir), diseñada específicamente para no forzar a los clientes de UI a implementar nada relacionado con las reglas de combate.
Comand	Combate/Atk/Comand.java	Interfaz mínima de un solo método (ejecutar); no obliga a AtaqueComand a exponer detalles internos del cálculo de daño.
CombateCallback	Combate/Atk/CombateCallBack.java	Define exactamente los cuatro métodos que AtaqueComand necesita de Combate (notificar ataque, notificar debilitado, notificar cambio, verificar estado), en vez de exponer toda la clase Combate al comando.

4.5 DIP — Principio de Inversión de Dependencias

Dependencia invertida	Ubicación	Evidencia
Entrenador → SelectorAtaque	Model/entrenador/Entrenador.java	Entrenador no depende de Scanner, de Swing ni de ninguna IA concreta; depende únicamente de la abstracción SelectorAtaque, inyectada vía setSelectorAtaque(...).
Combate → CombateObservador	Combate/Atk/Combate.java	Combate notifica a través de la interfaz CombateObservador; desconoce si el observador imprime en consola (BitacoraCombate) o actualiza Swing (CombateObservadorGUI).
AtaqueComand → CombateCallback	Combate/Atk/AtaqueComand.java	El comando depende de la abstracción CombateCallback (implementada por Combate) y no de la clase concreta Combate, lo que

Dependencia invertida	Ubicación	Evidencia
		reduce el acoplamiento entre el comando y el motor de batalla.
Pokemon → EstadoPokemon	Model/pokemon/Pokemon.java	Pokemon almacena y delega en la abstracción EstadoPokemon, nunca en una clase concreta de estado.

5. Principios GRASP — evidencias en el código

A continuación, se presentan evidencias de los patrones GRASP (General Responsibility Assignment Software Patterns) reconocibles en la asignación de responsabilidades del proyecto.

5.1 Experto en información (Information Expert)

Clase experta	Ubicación	Evidencia
Pokemon	Model/pokemon/Pokemon.java	Es quien posee los datos de hpActual y estado, por lo que es responsable de estaDesmayado() y de delegar iniciarTurno()/finalTurno(); ninguna otra clase decide si un Pokémon está desmayado.
Calculadora	Combate/Atk/Calculadora.java	Concentra el conocimiento de las fórmulas de daño porque necesita combinar datos de ambos Pokémon (atacante y defensor) y de la tabla de tipos; es el experto natural en ese cálculo.
TablaTipos	Combate/Atk/TablaTipos.java	Es la única clase que conoce las relaciones de efectividad entre TipoPokemon; Calculadora y AtaqueEspecial le consultan en vez de duplicar esa información.
MochilaGrupo	Patrones/composite/MochilaGrupo.java	Al contener la lista de ítems, es la experta en calcular getCantidad() total y en decidir tieneltem(...) recursivamente.

5.2 Creador (Creator)

Relación de creación	Ubicación	Evidencia
PokemonBuilder crea Pokemon	Patrones/builder/PokemonBuilder.java	build() ensambla la instancia con los datos acumulados; es la única vía recomendada para construir un Pokemon con estado inicial.
Entrenador crea su MochilaGrupo	Model/entrenador/Entrenador.java	El constructor de Entrenador inicializa this.mochila = new MochilaGrupo(...), ya que el entrenador contiene y usa directamente su mochila.

Relación de creación	Ubicación	Evidencia
Combate crea CombateMemento y AtaqueComand	Combate/Atk/Combate.java	Combate contiene los datos necesarios (turno, entrenadores, ataque elegido) para crear ambos objetos en el momento oportuno.

5.3 Bajo acoplamiento (Low Coupling) y Alta cohesión (High Cohesion)

El uso combinado de Facade, Strategy y Observer mantiene el acoplamiento bajo entre capas: la GUI depende de PokemonGameFacade y no de Pokedex/Entrenador/Combate directamente; Combate depende de interfaces (CombateObservador, SelectorAtaque vía Entrenador) y no de clases concretas de la UI. En cuanto a cohesión, clases como Calculadora, TablaTipos, ConsolaObservador y BitacoraCombate concentran una única razón de ser, evidenciando alta cohesión interna (coincide con las evidencias de SRP de la sección 4.1).

5.4 Controlador (Controller)

La clase Combate cumple el rol de controlador de caso de uso: coordina el ciclo completo de un combate (iniciarBatalla(), ejecutarAccion(), verificarEstadoPokemon()), sin ser ella misma la interfaz de usuario ni una entidad de dominio pura. Recibe las notificaciones de AtaqueComand implementando CombateCallback, y reenvía los eventos correspondientes a los observadores registrados, actuando como intermediario entre la capa de presentación (a través de los observadores) y las entidades de dominio (Entrenador, Pokemon).

5.5 Polimorfismo (Polymorphism)

Se evidencia en Ataque.atacar(Pokemon, Pokemon), redefinido de forma distinta en AtaqueFisico (daño directo) y AtaqueEspecial (con carga de turnos, efectividad de tipo y posibilidad de aplicar un estado alterado); y en EstadoPokemon.alIniciarTurno(...)/alFinalTurno(...), cuyo comportamiento cambia completamente según el estado concreto sin que el código cliente (Pokemon, Combate) use ninguna estructura condicional para diferenciarlos.

5.6 Fabricación pura (Pure Fabrication)

Clase fabricada	Ubicación	Justificación
AtaqueComand	Combate/Atk/AtaqueComand.java	No representa un concepto del dominio Pokémon en sí (un entrenador no piensa en "comandos"); se fabrica para encapsular la ejecución de un ataque como objeto y desacoplar Combate de la lógica interna de cada ataque.
CombateMemento	Combate/Atk/CombateMemento.java	Estructura de datos fabricada exclusivamente para permitir el snapshot/restore del combate, sin corresponder a ninguna entidad natural del juego.

Clase fabricada	Ubicación	Justificación
TablaTipos	Combate/Atk/TablaTipos.java	Fabricada para centralizar el conocimiento de efectividad entre tipos y evitar que esa responsabilidad recaiga en Pokemon o en Ataque, manteniendo alta cohesión.

5.7 Variación protegida (Protected Variation)

El patrón State protege a Pokemon y a Combate de la variación en las reglas de cada estado alterado (si mañana se agrega EstadoCongelado, ningún código existente se modifica). De forma análoga, el patrón Strategy (SelectorAtaque) protege a Entrenador y Combate de la variación en el mecanismo de entrada del jugador (consola, Swing, IA), y el patrón Observer protege a Combate de la variación en cómo se presentan los eventos de combate al usuario final.

6. Tabla de Antipatrones evitados

Antipatrón evitado	Como se evito	Patron aplicado
God Object	La lógica se distribuye entre clases como Pokedex, Combate, PokemonGameFacade y Paneles GUI, evitando una clase con todas las responsabilidades.	Facade, State, Observer, Command
Código duplicado	Los comportamientos reutilizables se encapsulan en clases de ataques y estados.	State, Command
Acoplamiento fuerte	Las clases interactúan mediante interfaces y abstracciones.	Observer, Facade
Condicionales extensos	Los comportamientos cambian mediante objetos de estado en lugar de múltiples if/switch.	State
Acceso global descontrolado	La Pokedex mantiene una única instancia controlada.	Singleton
Creación rígida de objetos	Los objetos pueden clonarse sin recrearlos manualmente.	Prototype
Dependencia directa de la interfaz	La GUI se comunica mediante la fachada y observadores.	Facade, Observer
Cambios difíciles de mantener	Las responsabilidades están separadas en paquetes y clases especializadas.	SOLID + Patrones

7. Prueba de Ejecución

https://drive.google.com/drive/folders/1I01G3rRfJ3Z_FiGek4vWiDtQXCmLdEiw?usp=sharing

8. Conclusiones

El proyecto PokemonGame evidencia un uso deliberado y bien documentado de patrones de diseño GoF: se identificaron con claridad tres patrones creacionales (Builder, Singleton, Prototype), dos estructurales (Facade, Composite) y cinco de comportamiento (State, Observer, Command, Memento, Strategy), varios de ellos trabajando en conjunto de forma coherente — por ejemplo, Memento se apoya en Prototype para clonar en profundidad a los entrenadores, y Builder se combina con Prototype para clonar ataques y con Command para desacoplar su ejecución del motor de combate.

En cuanto a los principios SOLID, el diseño demuestra especial cuidado en OCP y LSP a través de las jerarquías EstadoPokemon, ItemMochila y SelectorAtaque, todas ellas explícitamente pensadas para permitir extensión sin modificar código existente. La aplicación de DIP mediante interfaces como CombateObservador, SelectorAtaque y CombateCallback reduce sensiblemente el acoplamiento entre el motor de combate, el modelo de dominio y la capa de presentación en Swing.

Desde la perspectiva de GRASP, la asignación de responsabilidades sigue de forma consistente los criterios de Experto en información (Calculadora, TablaTipos, Pokemon) y Fabricación pura (AtaqueComand, CombateMemento, TablaTipos), lo que favorece tanto la cohesión de cada clase como el bajo acoplamiento general del sistema.